Cours 2

Éditeur de code et git local

1re année géomatique
2027-2028

Les fichiers du cours

Une archive par séance, un dossier par TD. Aujourd'hui, tout est dans `cours2/`.

```
cours2/                  un dossier par TD, dans l'ordre de la séance
├── 2a_vscode/
├── 2b_erreurs/
└── 2c_depot_altitudes/
```

Dans chaque dossier, la feuille du TD et son guide détaillé, en PDF et en page web.

L’éditeur de code

Ce qu’un éditeur de code ajoute à un éditeur de texte.

Rappel du cours 1 : édition et exécution

Explorateur de fichiers

travail

…\1c_programme\travail

- altitudes.py
- commandes.sh

ouvrir →

Éditeur de texte Bloc-notes, Notepad++

altitudes.py - Notepad++

```
1 altitudes = [79.6, 103.2, 98.3]
2 total = 0
3 for altitude in altitudes:
4     total = total + altitude
5 moyenne = total / len(altitudes)
6 print(f"moyenne : {moyenne:.1f} m")
```

pour un programme de plusieurs fichiers

↓ enregistrer, puis lancer

Terminal Git Bash

Git Bash

```
eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1/1c_programme/travail
$ python altitudes.py
moyenne : 93.7 m
```

> Un programme écrit dans LibreOffice Writer est enregistré en `.odt`, et `python` ne peut pas exécuter ce fichier.

Les fonctions d'un IDE

Un IDE réunit en une fenêtre les outils du cours 1, pour programmer sans passer de l'un à l'autre. Il ajoute d'autres aides, comme la complétion et le débogueur.

Fichiers

au cours 1 : l'explorateur

- arborescence du dossier
- recherche dans les fichiers

Éditeur

au cours 1 : Bloc-notes, Notepad++

aide syntaxique : coloration | soulignement des fautes | indentation | complétion

Terminal

au cours 1 : Git Bash

- bouton d'exécution

En option

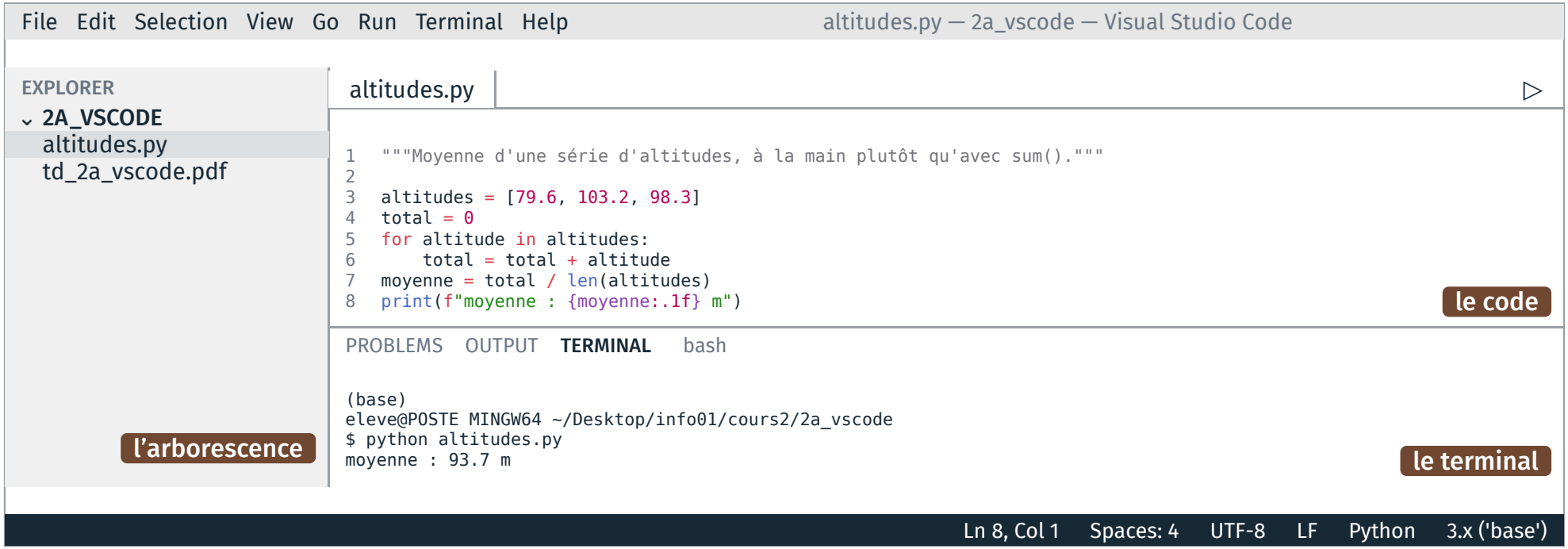
fenêtres de Spyder, par exemple

- débogueur
- figures

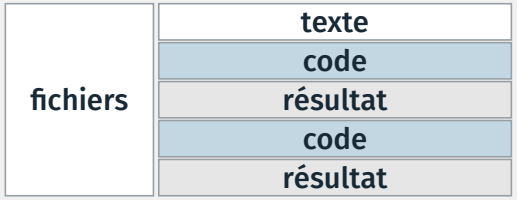
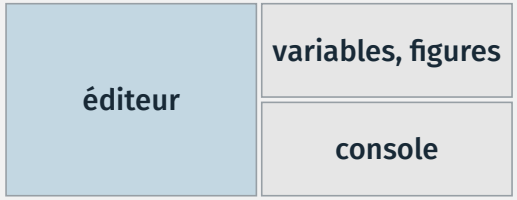
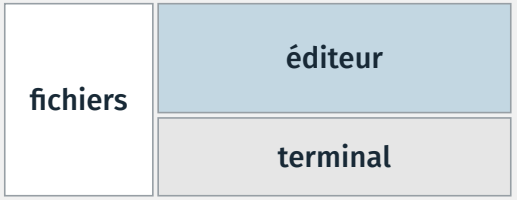
IDE : *integrated development environment*, environnement de développement intégré. Python doit être installé à part de VS Code, puis sélectionné dans l'éditeur (TD 2a).

Visual Studio Code

L'éditeur du module a trois zones : l'arborescence du dossier ouvert, le fichier, et le terminal, qui part de ce dossier.



Comparaison : JupyterLab, Spyder, VS Code

	JupyterLab (notebooks)	Spyder	VS Code
			
Tâche	suivre un cours et exécuter ses exemples un à un	écrire un programme de calcul, et regarder ses variables et ses figures	écrire un programme de plusieurs fichiers, suivi avec git
Fichiers `.py`	éditeur simple, exécution dans un terminal de JupyterLab	exécution par `F5`, ou par cellules `# %%`	exécution avec l'extension Python
Notebooks `.ipynb`	ouverture et exécution	conversion en fichier `.py`, ou plugin `spyder-notebook`	ouverture et exécution avec l'extension Jupyter
Autres langages	par un autre noyau (R, Julia)	aucun	une extension par langage

Le module emploie JupyterLab et VS Code. D'autres cours emploient Spyder.

Aide syntaxique : coloration, soulignement

Une extension de l'éditeur contient les règles de syntaxe du langage. D'après ces règles, l'éditeur colore chaque catégorie de mot, sans modifier le fichier.

Le fichier d'un programme

```
altitude = 79.6
print(altitude)
```

Affiché par l'éditeur de code

```
altitude = 79.6
print(altitude)
```

Il souligne de même ce qui ne suit pas ces règles, sans rien lancer.

```
altitudes = [79.6, 103.2]
for altitude in altitudes
    print(altitude)
```

Le deux-points manque. Sans l'éditeur, la faute n'apparaîtrait qu'au lancement : `SyntaxError: expected ':'`.

Aide syntaxique : indentation

Un espace et une tabulation sont deux caractères différents. Une tabulation vaut le nombre de colonnes que l'éditeur lui donne, et ce réglage change d'un éditeur à l'autre.

Tabulation réglée sur 4 colonnes

```
def surface(longueur, largeur):
····aire = longueur * largeur
→   return aire
```

les deux lignes semblent alignées

Le même fichier, tabulation sur 8

```
def surface(longueur, largeur):
····aire = longueur * largeur
→       return aire
```

le décalage apparaît

· marque un espace, → une tabulation, comme l'éditeur les dessine. Les octets du fichier sont les mêmes des deux côtés : seul le réglage de l'éditeur change.



Dans VS Code, la touche Tab insère quatre espaces, et la barre d'état affiche « Spaces: 4 ». Un clic sur cette mention propose de convertir les tabulations du fichier en espaces.

TRAVAUX DIRIGÉS

TD 2a Configurer VS Code

`cours2/2a_vscode/`

≈ 25′

TRAVAUX DIRIGÉS

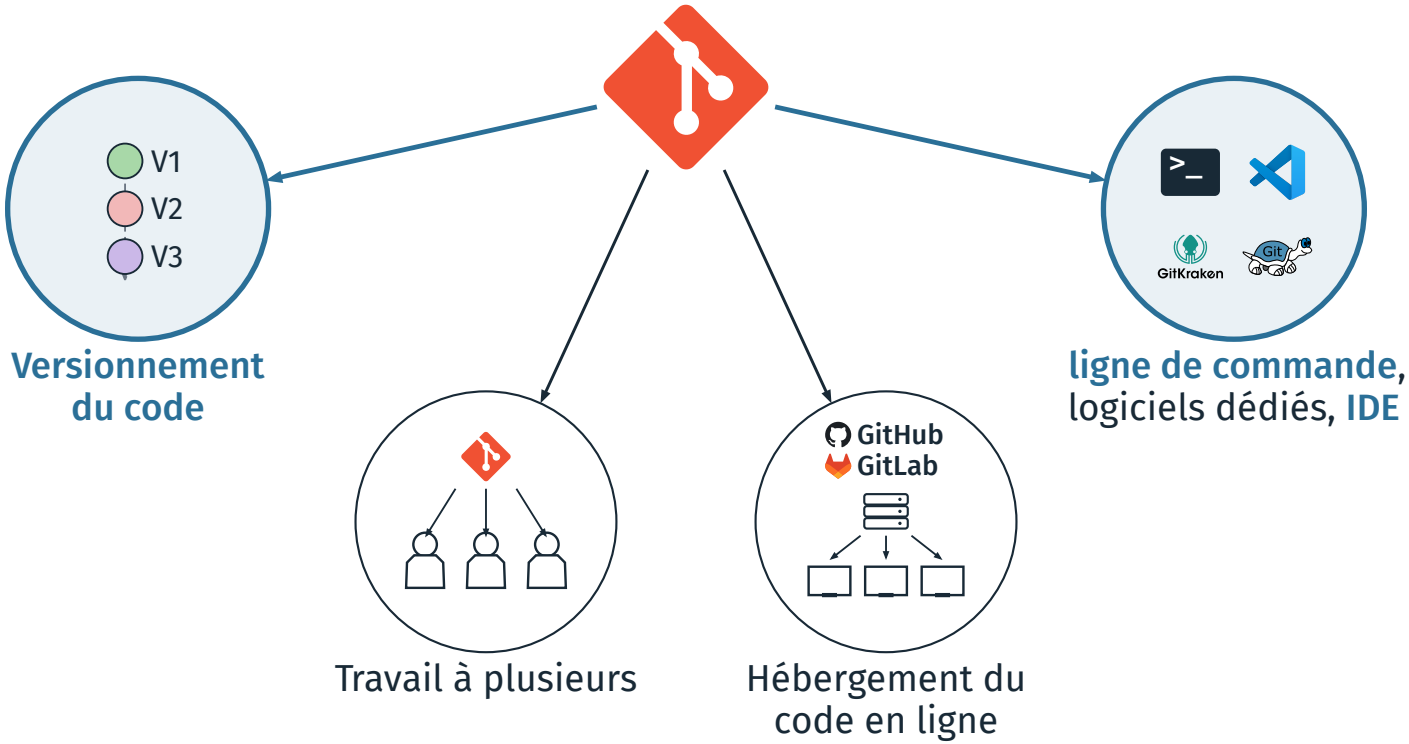
TD 2b Trois programmes fautifs ≈ 10′

`cours2/2b_erreurs/`

Git local

Enregistrer les versions successives d’un projet, les comparer, travailler sur une variante et la fusionner.

Git : vue d'ensemble



Les parties en couleur sont traitées dans cette séance : le versionnement du code, en ligne de commande et dans un IDE. Le travail à plusieurs et l'hébergement du code en ligne sont pratiqués au cours 6.

Git : fonctionnalités

- Enregistrement des versions successives du projet

Sans git
une copie
par version

```
altitudes.py
altitudes_v2.py
altitudes_final.py
altitudes_final_OK.py
```

Chaque flèche part d'un commit enfant et pointe vers son parent.

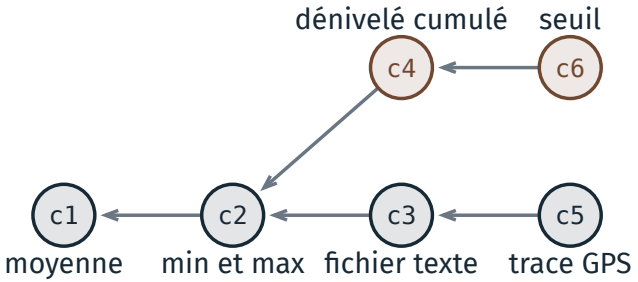
Avec git
un commit
par version



Exemple tiré des TD 2c, 3a, 3b et 4a : script `altitudes.py`, puis `stats_trace.py`.

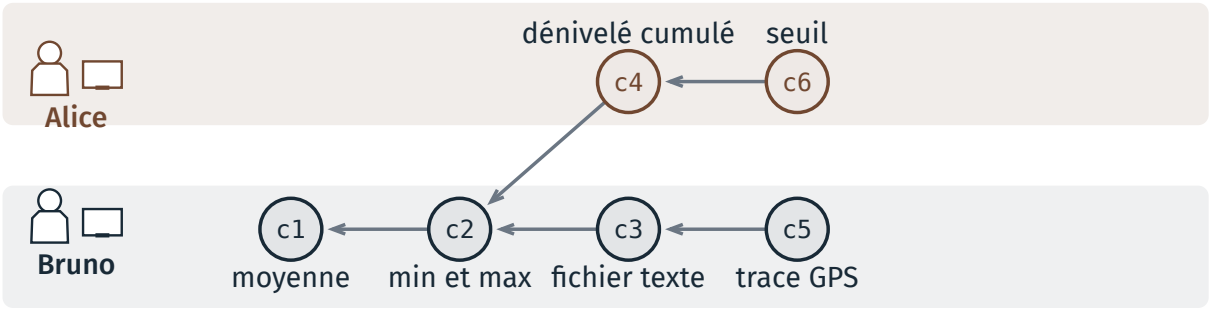
Git : fonctionnalités

- Enregistrement des versions successives du projet
- Modifications en parallèle, sur des branches



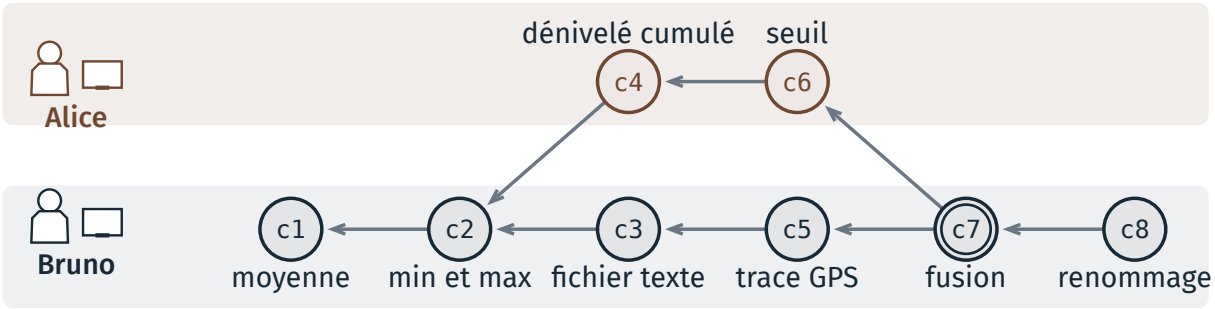
Git : fonctionnalités

- Enregistrement des versions successives du projet
- Modifications en parallèle, sur des branches
- Une branche par fonctionnalité, créée par le même développeur ou par un autre



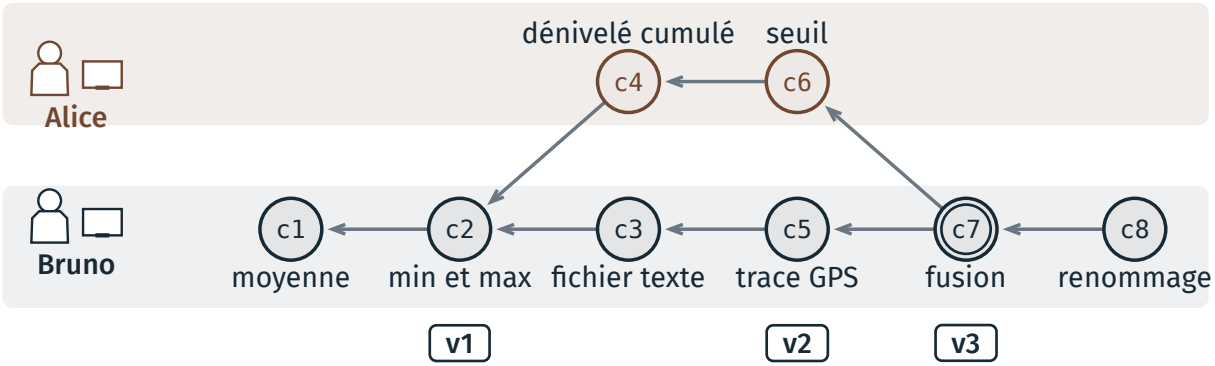
Git : fonctionnalités

- Enregistrement des versions successives du projet
- Modifications en parallèle, sur des branches
- Une branche par fonctionnalité, créée par le même développeur ou par un autre
- Fusion de deux branches



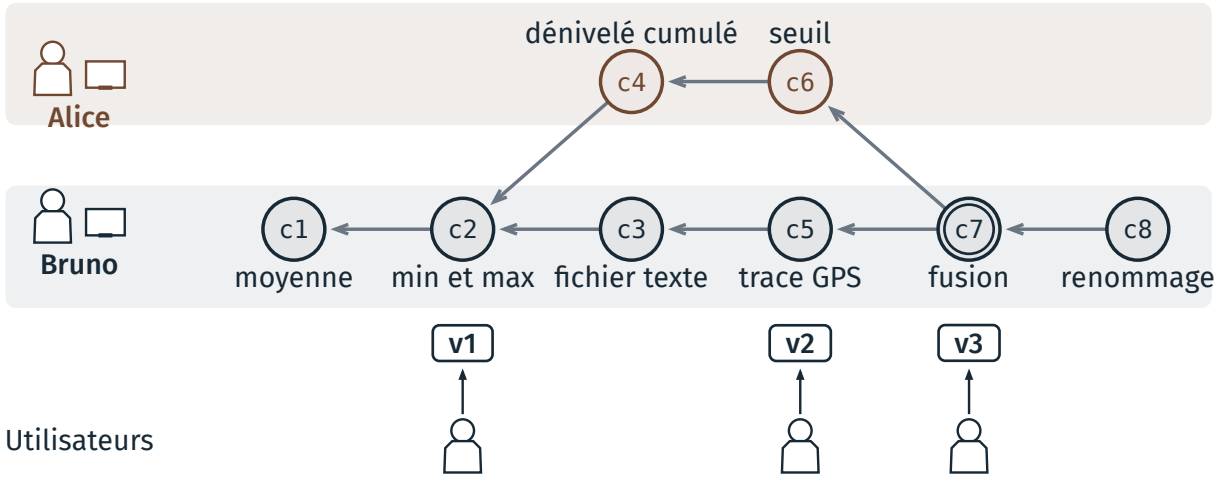
Git : fonctionnalités

- Enregistrement des versions successives du projet
- Modifications en parallèle, sur des branches
- Une branche par fonctionnalité, créée par le même développeur ou par un autre
- Fusion de deux branches
- Étiquette sur certaines versions



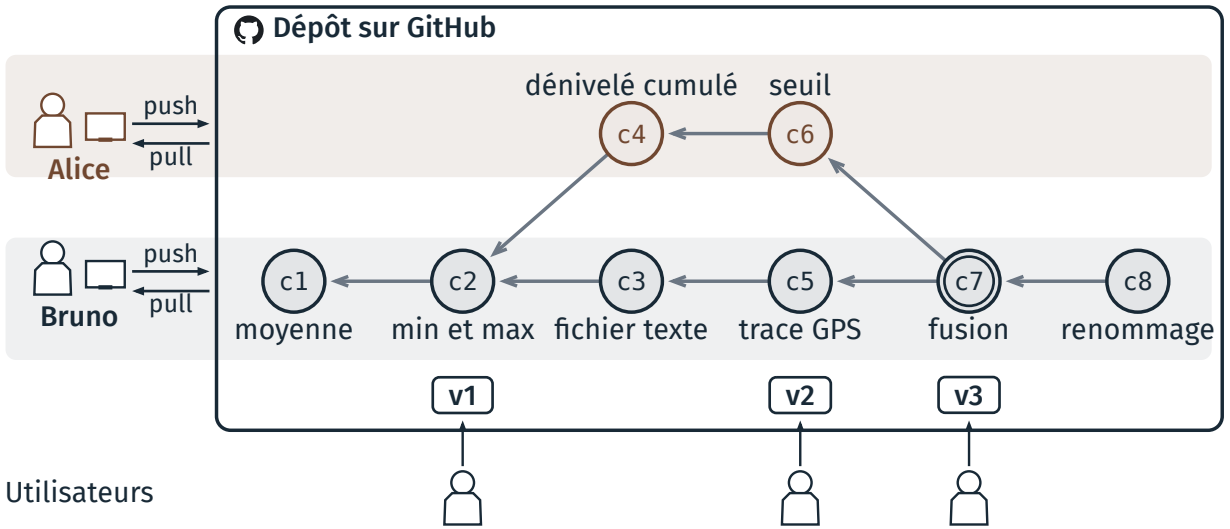
Git : fonctionnalités

- Enregistrement des versions successives du projet
- Modifications en parallèle, sur des branches
- Une branche par fonctionnalité, créée par le même développeur ou par un autre
- Fusion de deux branches
- Étiquette sur certaines versions
- Accès des utilisateurs aux versions étiquetées



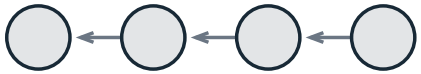
Git : fonctionnalités

- Enregistrement des versions successives du projet
- Modifications en parallèle, sur des branches
- Une branche par fonctionnalité, créée par le même développeur ou par un autre
- Fusion de deux branches
- Étiquette sur certaines versions
- Accès des utilisateurs aux versions étiquetées
- Partage du dépôt sur une forge



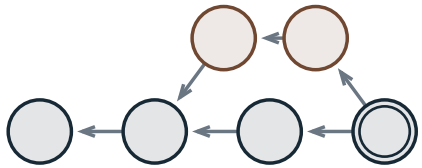
Git : cas d’usage

En projet seul



- Sauvegarde de chaque version du code
- Mise à disposition des versions terminées, si besoin
- Historique simple, souvent sur une seule branche

En projet à plusieurs



- Mise en commun du travail de chacun
- Travail en parallèle sur des branches, sans gêner les autres
- Règles communes pour les branches et les messages de commit

Au TD 4a, chacun versionne son projet seul. Au cours 6 et au projet du cours 7, le dépôt est partagé à plusieurs.

La commande git

git est un programme en ligne de commande. Le mot qui suit `git` est une sous-commande, qui a ses propres options et ses propres arguments.

Commande tapée	Sous-commande	Options	Arguments
`git init`	`init`		
`git add altitudes.py`	`add`		`altitudes.py`
`git commit -m "Ajoute le dénivelé cumulé"`	`commit`	`-m "Ajoute le dénivelé cumulé"`	
`git log --oneline`	`log`	`--oneline`	

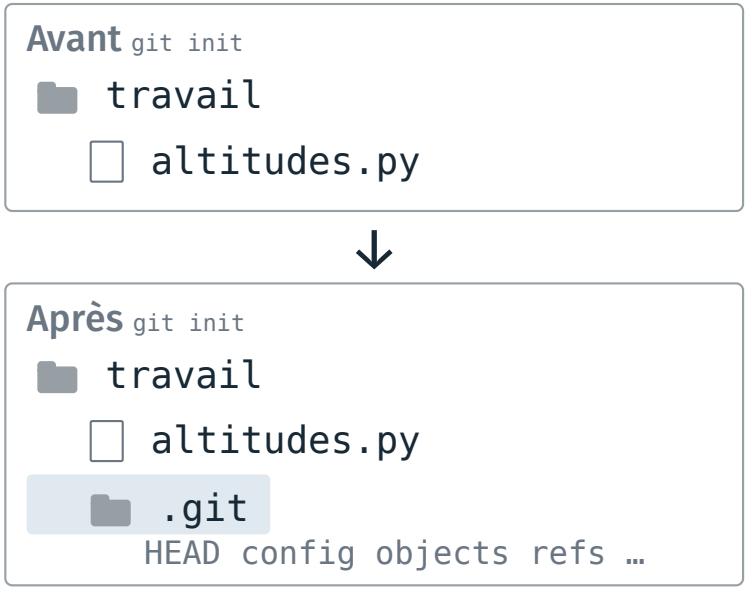
Forme : `git <sous-commande> [options] <arguments>`, celle des commandes du cours 1 avec un mot de plus. `-m` prend une valeur : le message.

`git --help`	liste les sous-commandes courantes
`git add -h`	affiche les options de `git add`
`git add --help`	ouvre la page du manuel de `git add`, dans le navigateur sous Windows

Créer un dépôt : git init

`git init` fait du dossier courant un dépôt : il y crée un dossier caché, `.git`, qui contiendra l'historique.

```
$ cd ~/Desktop/info01/cours2/2c_depot_altitudes/travail
$ pwd
/c/Users/eleve/Desktop/info01/cours2/2c_depot_altitudes/travail
$ git init
Dépôt Git vide initialisé dans …/2c_depot_altitudes/travail/.git/
$ ls -a
.
..
altitudes.py
.git
```



Les fichiers du dossier ne changent pas. Supprimer `.git` supprime l'historique, et laisse les fichiers.

L’auteur des commits : git config

Chaque commit enregistre le nom et l’adresse de son auteur. Sans eux, `git commit` s’arrête sur « Identité d’auteur inconnue ».

```
$ git config user.name "Alice Martin"
$ git config user.email alice.martin@ensg.eu
$ git commit -m "Ajoute le calcul de la moyenne"
…
$ git log
commit 64ed21360dbd4701ade422da243da0fb242d008a
Author: Alice Martin <alice.martin@ensg.eu>
Date:   Mon Sep 21 16:14:20 2026 +0200

    Ajoute le calcul de la moyenne
```

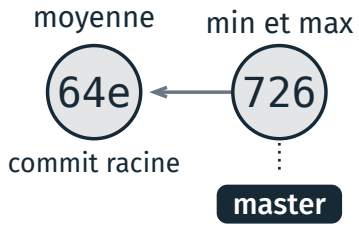
	`git config`	`git config --global`
S’applique à	ce dépôt	tous les dépôts du compte
Fichier	`.git/config`	`~/.gitconfig`
En salle	à employer	à éviter, car le compte est partagé

Le nom et l’adresse indiquent qui a fait chaque modification, et à qui poser une question. git les accepte sans vérification. Le réglage du dépôt l’emporte sur celui du compte. Au cours 6, la forge demande une authentification pour recevoir des commits.

Enregistrer une modification : add, puis commit

`git add` ajoute un fichier à la prochaine version ; `git commit` enregistre cette version, avec un message.

```
$ git status
$ git add altitudes.py
$ git status
$ git commit -m "Ajoute le calcul de la moyenne"
[master (commit racine) 64ed213] Ajoute le calcul de la moyenne
$ git status
…
$ git log --oneline
726d93f Remplace la moyenne par les altitudes extrêmes
64ed213 Ajoute le calcul de la moyenne
```



Avant `git add`

```
Fichiers non suivis:
  altitudes.py
```

Après `git add`

```
Modifications qui seront validées :
  nouveau fichier : altitudes.py
```

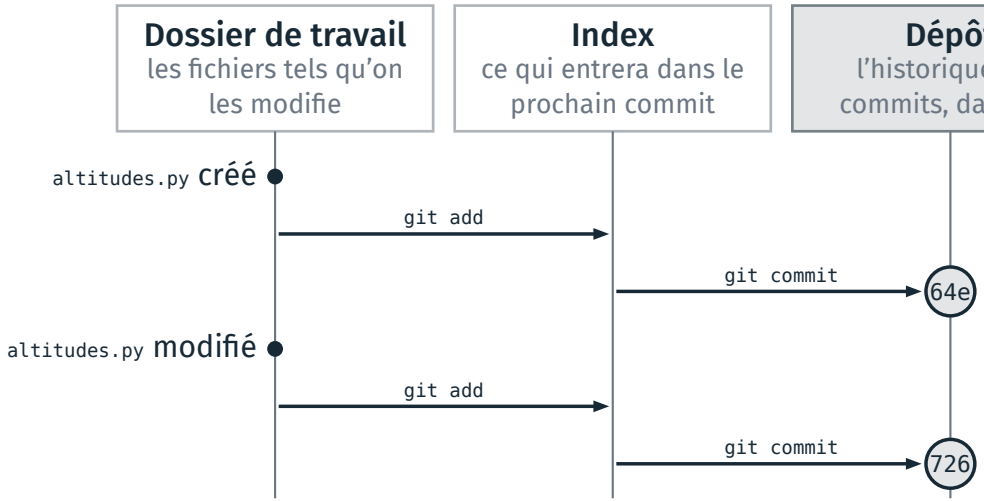
Après `git commit`

```
rien à valider, la copie de
travail est propre
```

`64ed213` est le début de l'identifiant du commit, calculé à partir de son contenu ; `master` désigne le dernier commit, et avance à chaque commit.

Trois zones : dossier, index, dépôt

Un commit enregistre ce que `git add` a placé dans l'index, et rien d'autre.



Sortie de `git status`

« Fichiers non suivis »

« Modifications qui seront validées »

« rien à valider, la copie de travail est propre »

« Modifications qui ne seront pas validées »

« Modifications qui seront validées »

« rien à valider, la copie de travail est propre »

Faire des commits souvent : git ne peut revenir qu'à une version enregistrée par un commit. `git status` affiche aussi la branche courante et les commandes qui s'appliquent. La taper avant toute autre, et avant de demander de l'aide.

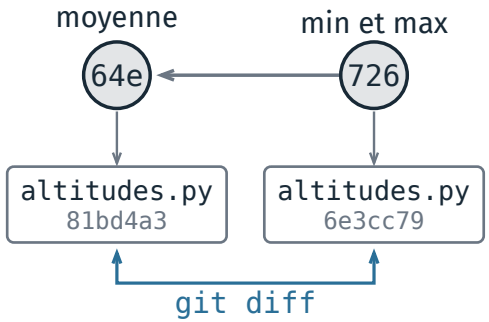
Ce qui a changé : git diff

`git diff` montre les lignes modifiées depuis le dernier commit : en `-` l'ancienne version, en `+` la nouvelle, avec trois lignes de contexte.

```
$ git diff
diff --git a/altitudes.py b/altitudes.py
index 81bd4a3..6e3cc79 100644
--- a/altitudes.py
+++ b/altitudes.py
@@ -1,8 +1,6 @@
-"""Moyenne d'une série d'altitudes, à la main plutôt qu'avec sum()."""
+"""Altitudes extrêmes d'une série de mesures."""

 altitudes = [79.6, 103.2, 98.3]
-total = 0
-for altitude in altitudes:
-    total = total + altitude
-moyenne = total / len(altitudes)
-print(f"moyenne : {moyenne:.1f} m")
+print(f"altitude minimale : {min(altitudes):.1f} m")
+print(f"altitude maximale : {max(altitudes):.1f} m")
+print(f"max - min : {max(altitudes) - min(altitudes):.1f} m")
```

Contenu des commits



À chaque commit, git enregistre le contenu complet des fichiers ajoutés ou modifiés, et réutilise celui des fichiers inchangés. `git diff` calcule l'écart entre deux contenus quand on le tape.

Sur la ligne `index`, git affiche les identifiants des deux contenus comparés. `@@ -1,8 +1,6 @@` : le passage commence à la ligne 1, et fait 8 lignes avant et 6 lignes après. VS Code montre le même écart côte à côte.

Annuler une modification : git restore, git revert

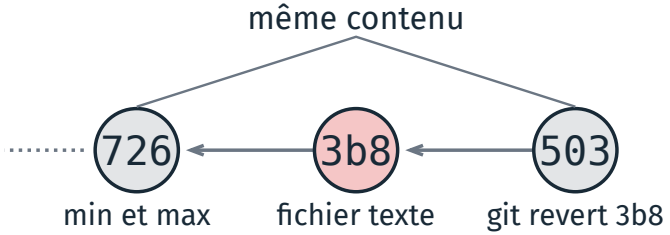
Avant un commit, `git restore` remet le fichier dans l'état du dernier commit, et la modification est perdue. Après un commit, `git revert` le défait par un nouveau commit.

Modification non validée

```
$ git diff
@@ -2,5 +2,5 @@
-print(f"altitude maximale : {max(altitudes):.1f} m")
+print(f"altitude maximale : {max(altitude):.1f} m")
```

```
$ git restore altitudes.py
$ git status
Sur la branche master
rien à valider, la copie de travail est propre
```

Commit déjà fait



À gauche, une ligne abîmée par erreur est remise en état. À droite, `3b8` reste dans l'historique, et `503` remet le contenu de `726`.

Ignorer des fichiers : .gitignore

Un fichier `.gitignore` liste des motifs de noms. `git status` n'affiche plus les fichiers qui y correspondent, et `git add` ne les ajoute pas.

Ce qu'on ne versionne pas	Pourquoi	Motif
les notes personnelles	elles ne font pas partie du programme	`notes.txt`
les fichiers produits par un programme	ils se refont depuis leur source	`__pycache__/`
les données de travail	chaque version resterait dans l'historique	`donnees/`
les mots de passe et les clés	toute copie du dépôt les contiendrait	`.env`

`.gitignore`

```
notes.txt
```

`ls`

```
altitudes.py
notes.txt
README.md
```

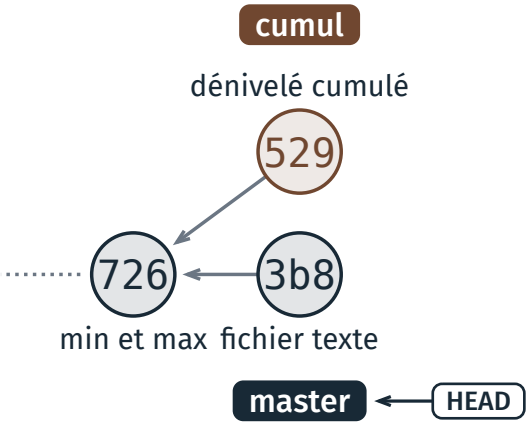
`git status`

```
Sur la branche master
rien à valider, la copie de
travail est propre
```

`.gitignore` se versionne avec le projet.

Les branches

Une branche est un nom posé sur un commit. Elle avance d'un commit à chaque commit fait sur elle ; `HEAD` désigne la branche courante.



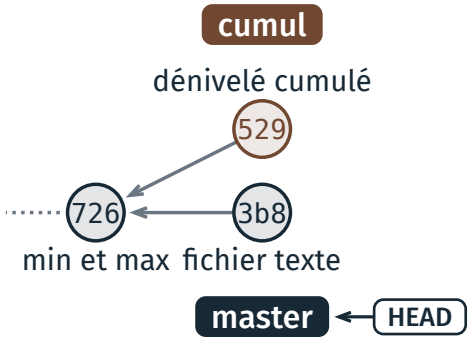

```
$ git checkout -b cumul
Basculement sur la nouvelle branche 'cumul'
…
$ git checkout master
…
$ git log --oneline --graph --all --decorate
* 3b88896 (HEAD -> master) Lit les altitudes dans un fichier texte
| * 5298338 (cumul) Ajoute le dénivelé cumulé
|/
* 726d93f Remplace la moyenne par les altitudes extrêmes
* 64ed213 Ajoute le calcul de la moyenne
```

`git checkout -b nom` crée une branche et s'y place ; `git checkout nom` change de branche ; `git branch` les liste.

Changer de branche change les fichiers

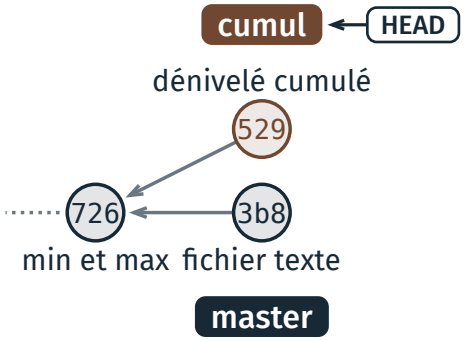
`git checkout` remplace les fichiers du dossier par ceux du dernier commit de la branche choisie.

Sur master




```
$ python altitudes.py
…
max - min : 23.6 m
```

Après `git checkout cumul`



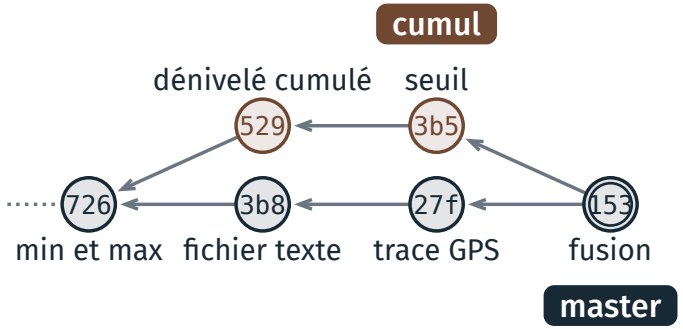

```
$ python altitudes.py
…
max - min : 23.6 m
cumul en hausse : 23.6 m
cumul en baisse : 4.9 m
```

Sous chaque graphe, la fin de ce qu'affiche le programme. Le cumul n'existe que sur la branche `cumul`.

Fusionner deux branches : git merge, git rebase

`git merge` apporte dans la branche courante les commits d'une autre branche. `git rebase` refait les commits de la branche courante à la suite du dernier commit d'une autre branche.

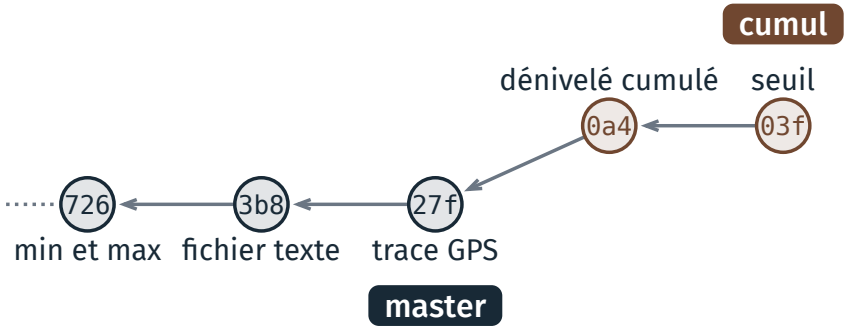
Avec `git merge`




```
$ git merge cumul -m "Fusionne le dénivelé cumulé"
Fusion automatique de altitudes.py
Merge made by the 'ort' strategy.
```

`153`, le commit de fusion, a deux parents (double trait).

Avec `git rebase`

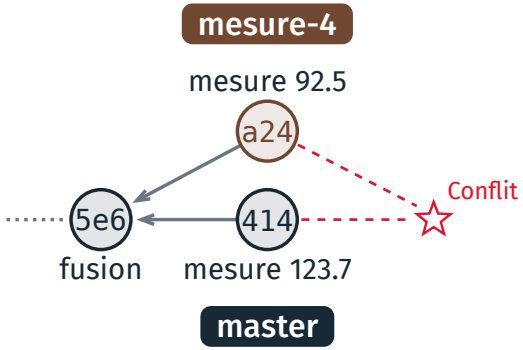



```
$ git checkout cumul
$ git rebase master
Rebasage et mise à jour de refs/heads/cumul avec succès.
```

git refait `529` et `3b5` en `0a4` et `03f`, à la suite de `27f`.

Un conflit de fusion

Quand les deux branches ont modifié la même ligne, git arrête la fusion et écrit les deux versions dans le fichier.



Commit	La troisième ligne d'`altitudes.py`
`5e6`, avant	`altitudes = [79.6, 103.2, 98.3]`
`a24`, `mesure-4`	`altitudes = [79.6, 103.2, 98.3, 92.5]`
`414`, `master`	`altitudes = [79.6, 103.2, 98.3, 123.7]`

```
$ git merge mesure-4
Fusion automatique de altitudes.py
CONFLIT (contenu) : Conflit de fusion dans altitudes.py
```

`mesure-4` et `master` ont chacune ajouté une mesure à la même ligne, modifiée des deux côtés depuis `5e6`. La partie modifiée est soulignée.

Résoudre un conflit

Écrire la ligne voulue à la place des deux versions et des marqueurs, puis `git add` et `git commit` terminent la fusion.

`altitudes.py` **après** `git merge`

```
<<<<<<< HEAD
altitudes = [79.6, 103.2, 98.3, 123.7]
=======
altitudes = [79.6, 103.2, 98.3, 92.5]
>>>>>>> mesure-4
```

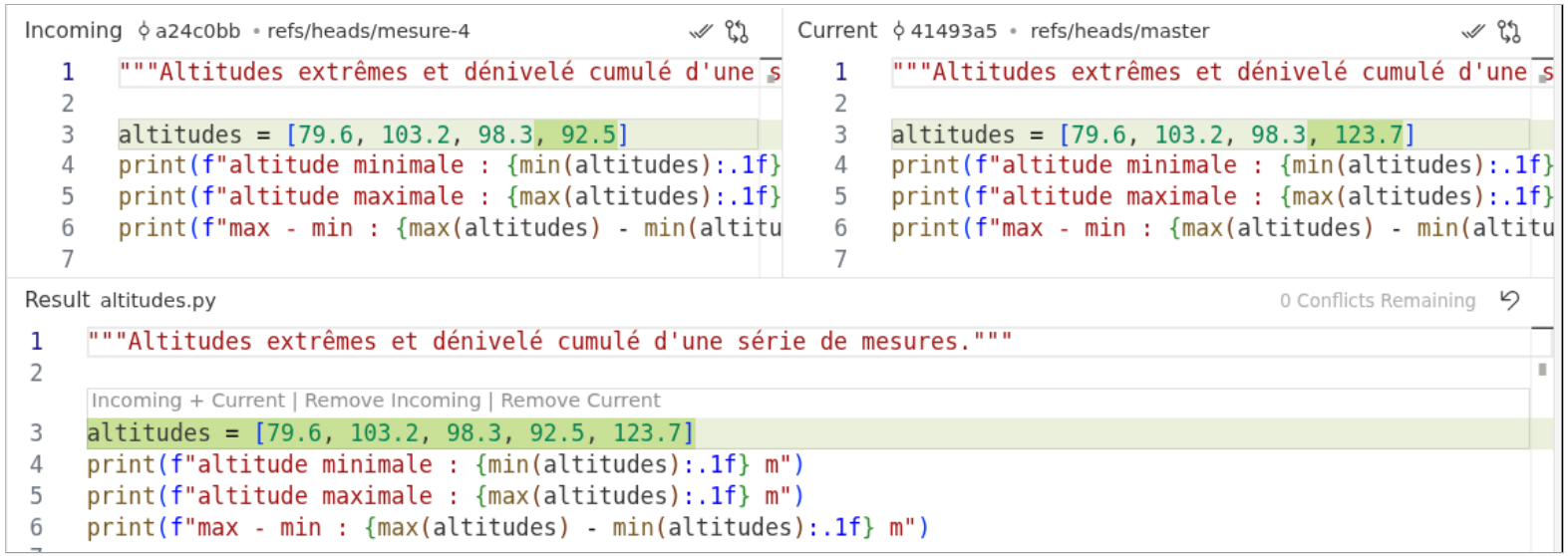
La ligne écrite, puis la fin de la fusion

```
altitudes = [79.6, 103.2, 98.3, 92.5, 123.7]
$ git add altitudes.py
$ git commit -m "Fusionne les deux mesures"
[master e0c82f8] Fusionne les deux mesures
```

Entre `<<<<<<< HEAD` et `=======`, la ligne de la branche courante ; entre `=======` et `>>>>>>>`, celle de la branche fusionnée. La bonne ligne garde les deux mesures.

Résoudre un conflit dans VS Code

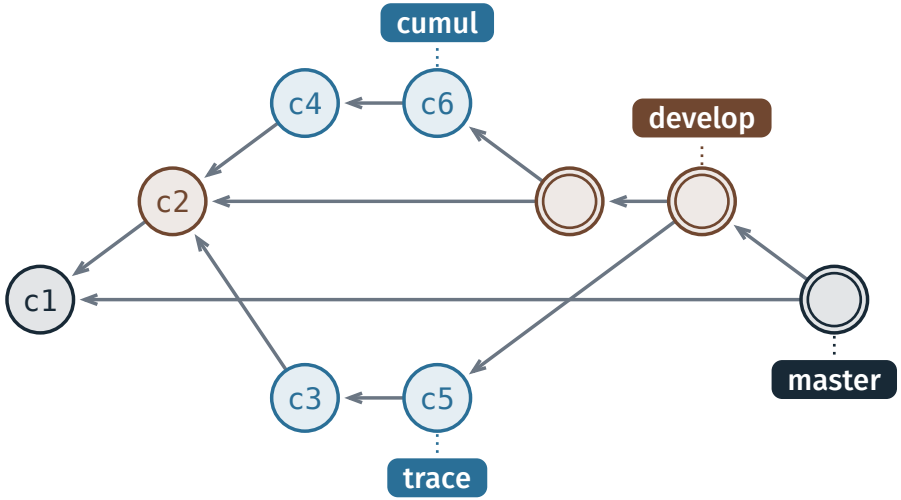
VS Code ouvre son éditeur de fusion par le bouton « Resolve in Merge Editor », en bas à droite du fichier en conflit.



Incoming montre la version de `mesure-4`, la branche fusionnée, et Current celle de `master`, la branche courante. Après un clic sur « Accept Combination (Incoming First) », VS Code écrit dans Result la ligne de la diapositive précédente. Avec « Complete Merge », en bas à droite, il enregistre le fichier et fait le `git add` ; `git commit` termine la fusion.

L’organisation des branches à plusieurs

`master` ne reçoit que les versions terminées. `develop` réunit le travail en cours ; chaque tâche se fait sur sa propre branche, fusionnée dans `develop` quand elle est terminée.

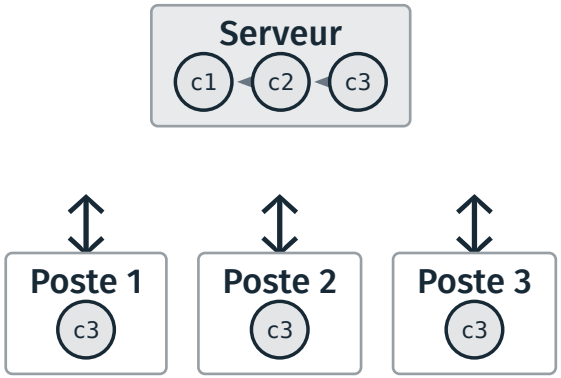


Les commits sont ceux de « Git : fonctionnalités » : Alice écrit le dénivelé cumulé et le seuil, Bruno la lecture du fichier texte et de la trace GPS, chacun dans sa copie du dépôt. Deux fusions réunissent les tâches dans `develop`, et une troisième porte la version terminée dans `master`.

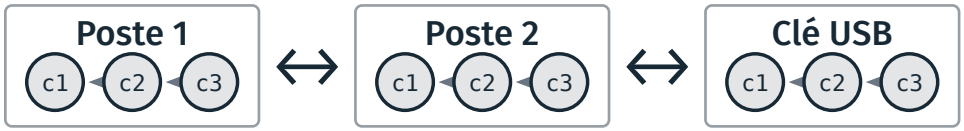
Les systèmes de version

Un système centralisé garde l'historique sur un serveur. Dans un système distribué, comme git, chaque copie du projet contient tout l'historique.

Centralisé : CVS (1986), Subversion (2000)



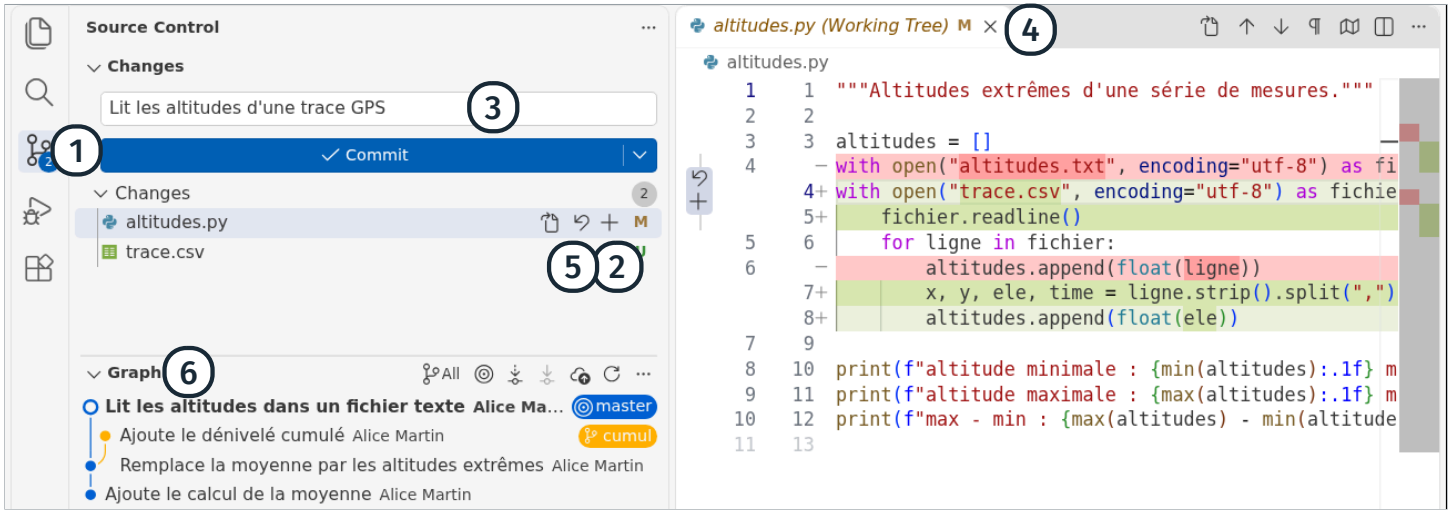
Distribué : git, Mercurial (2005)



À gauche, un poste n'a que la dernière version, et il faut le serveur pour faire un commit ou lire l'historique. À droite, chaque dépôt a tous les commits, et un commit se fait sans réseau.

git dans VS Code : le panneau Source Control

VS Code lance les commandes git depuis son panneau Source Control.



① panneau : `Ctrl` + `Maj` + `G`
② bouton + : `git add`
③ message et Commit : `git commit -m`
④ fichier ouvert : `git diff`
⑤ bouton ↶ : `git restore`
⑥ Graph : `git log --graph --all`

La capture précède le commit `27f` (trace GPS) : `altitudes.py` est modifié (M), et `trace.csv`, copié depuis `cours3/3a_denivele/depart/` (TD 3a), n'est pas suivi (U).

TRAVAUX DIRIGÉS

TD 2c Un dépôt git pour altitudes.py ≈ 40′

`cours2/2c_depot_altitudes/`

À retenir

Récapituler les commandes git de la séance, puis les raccourcis de VS Code.

Aide-mémoire : enregistrer des versions avec git



Créer un dépôt, enregistrer		Comparer, annuler, ignorer	
`git init`	crée le dépôt `.git`	`git diff`	montre les lignes modifiées
`git config user.name "Nom"`	nomme l'auteur des commits	`git restore fichier`	remet le fichier du dernier commit
`git status`	affiche l'état, à taper d'abord	`git revert commit`	défait un commit par un nouveau commit
`git add fichier`	place le fichier dans l'index	`.gitignore`	liste les fichiers que git ignore
`git commit -m "message"`	enregistre l'index dans un commit	`git commande -h`	affiche les options
`git log --oneline`	liste les commits	`git commande --help`	ouvre le manuel

Aide-mémoire : branches, fusion et conflit

Branches		Fusion	
`git branch`	liste les branches, `*` devant la branche courante	`git merge nom`	apporte les commits de `nom` dans la branche courante
`git checkout -b nom`	crée la branche `nom` et s'y place	`git rebase nom`	refait les commits de la branche courante à la suite de `nom`
`git checkout nom`	passe sur la branche `nom`, et change les fichiers du dossier	`git log --oneline --graph --all`	affiche le graphe de toutes les branches

```
<<<<<<< HEAD
altitudes = [79.6, 103.2, 98.3, 123.7]
=======
altitudes = [79.6, 103.2, 98.3, 92.5]
>>>>>>> mesure-4
```

Conflit	
`<<<<<<< HEAD`	ouvre la version de la branche courante
`>>>>>>> nom`	ferme la version de la branche fusionnée, après `=======`
`git add`, `git commit`	terminent la fusion, une fois la ligne voulue écrite à la place des marqueurs
`git merge --abort`	annule la fusion en cours

Aide-mémoire : VS Code

Raccourcis et menus		Réglages, Python et git	
`Ctrl` + `Maj` + `P`	ouvre la palette de commandes	User	règle VS Code pour le compte, dans `settings.json`
`Ctrl` + `,`	ouvre les réglages	Workspace	règle le dossier ouvert, dans `.vscode/settings.json`, et l'emporte sur User
`Ctrl` + `Maj` + `X`	ouvre les extensions	« Python: Select Interpreter »	choisit le Python de l'extension
Terminal, New Terminal	ouvre Git Bash dans le dossier ouvert	bouton ▶	lance le fichier Python ouvert
`Ctrl` + `Maj` + `G`	ouvre le panneau Source Control	bouton +, puis Commit	font `git add`, puis `git commit`
`Ctrl` + `Maj` + `V`	affiche l'aperçu d'un fichier Markdown	« Resolve in Merge Editor »	ouvre l'éditeur de fusion d'un fichier en conflit